

Ensamblador MIPS64

.text o .code		Comienzo de código
.data		Comienzo de datos
.org	DIR	Dirección de comienzo de ensamblaje
.space	N	Reserva N octetos de memoria
.byte	b1,b2,...	Inicializa una zona de memoria con octetos
.word16	w1,w2,...	Inicializa una zona de memoria con unidades de 16 bits
.word32	w1,w2,...	Inicializa una zona de memoria con unidades de 32 bits
.word	n1,n2,...	Inicializa una zona de memoria con unidades de 64 bits
.ascii	STRING	Inicializa una zona de memoria con STRING
.asciiz	STRING	Idéntica a la anterior, añadiendo NULL al final
.double	d1,d2,...	Inicializa una zona de memoria con números en c. f.

dadd	Rd,Rs,Rt	Suma (64 bits)	$Rd = Rs + Rt$
daddu	Rd,Rs,Rt	Suma (64 bits sin signo)	$Rd = Rs + Rt$
dsub	Rd,Rs,Rt	Resta (64 bits)	$Rd = Rs - Rt$
dsubu	Rd,Rs,Rt	Resta (64 bits sin signo)	$Rd = Rs - Rt$
dmul	Rd,Rs,Rt	Multiplica (64 bits)	$Rd = Rs * Rt$
dmulu	Rd,Rs,Rt	Multiplica (64 bits sin signo)	$Rd = Rs * Rt$
ddiv	Rd,Rs,Rt	Divide (64 bits)	$Rd = Rs / Rt$
ddivu	Rd,Rs,Rt	Divide (64 bits sin signo)	$Rd = Rs / Rt$
daddi	Rd,Rs,inm	Suma Inmediato (64 bits)	$Rd = Rs + inm$
daddui	Rd,Rs,inm	Suma inmediato (64 bits sin signo)	$Rd = Rs + inm$

and	Rd,Rs,Rt	And lógico (64 bits)	$Rd = Rs \text{ AND } Rt$
or	Rd,Rs,Rt	Or lógico (64 bits)	$Rd = Rs \text{ OR } Rt$
xor	Rd,Rs,Rt	Or exclusivo (64 bits)	$Rd = Rs \text{ XOR } Rt$
andi	Rt,Rs,inm	And lógico inmediato (64 bits)	$Rt = Rs \text{ AND } inm$
ori	Rt,Rs,inm	Or lógico inmediato (64 bits)	$Rt = Rs \text{ OR } inm$
xori	Rt,Rs,inm	Or exclusivo inmediato (64 bits)	$Rt = Rs \text{ XOR } inm$

lb	Rt,offset(Rs)	Carga (8 bits)	$Rt[0..7] = \text{MEM}[Rs + \text{offset}]$
lh	Rt,offset(Rs)	Carga (16 bits)	$Rt[0..15] = \text{MEM}[Rs + \text{offset}]$
lw	Rt,offset(Rs)	Carga (32 bits)	$Rt[0..31] = \text{MEM}[Rs + \text{offset}]$
ld	Rt,offset(Rs)	Carga (64 bits)	$Rt[0..63] = \text{MEM}[Rs + \text{offset}]$
lbu	Rt,offset(Rs)	Carga (8 bits sin signo)	$Rt[0..7] = \text{MEM}[Rs + \text{offset}]$
lhu	Rt,offset(Rs)	Carga (16 bits sin signo)	$Rt[0..15] = \text{MEM}[Rs + \text{offset}]$
lwu	Rt,offset(Rs)	Carga (32 bits sin signo)	$Rt[0..31] = \text{MEM}[Rs + \text{offset}]$
lui	Rt,imm	Carga un valor inmediato	$Rt = imm \parallel 0^{16}$

sb	Rt,offset(Rs)	Almacenamiento (8 bits)	$\text{MEM}[Rs + \text{offset}] = Rt[0..7]$
sh	Rt,offset(Rs)	Almacenamiento (16 bits)	$\text{MEM}[Rs + \text{offset}] = Rt[0..15]$
sw	Rt,offset(Rs)	Almacenamiento (32 bits)	$\text{MEM}[Rs + \text{offset}] = Rt[0..31]$
sd	Rt,offset(Rs)	Almacenamiento (64 bits)	$\text{MEM}[Rs + \text{offset}] = Rt[0..63]$

j	dest	Salto restringido a 256 MB	$PC = PC[63..28] \parallel index \parallel 0^2$
jr	Rs	Salto indirecto a registro	$PC = Rs$
jal	dest	Salto directo con retorno en R31	$PC = PC[63..28] \parallel index \parallel 0^2$
jalr	Rs	Salto indirecto con retorno en R31	$PC = Rs$

beq	Rs,Rt,dest	Salto restringido a ± 128 KB	Si $Rs = Rt \rightarrow PC = PC[63..18] + index \parallel 0^2$
bne	Rs,Rt,dest	Salto restringido a ± 128 KB.	Si $Rs \neq Rt \rightarrow PC = PC[63..18] + index \parallel 0^2$
beqz	Rs,dest	Salto restringido a ± 128 KB.	Si $Rs = 0 \rightarrow PC = PC[63..18] + index \parallel 0^2$
bnez	Rs,dest	Salto restringido a ± 128 KB.	Si $Rs \neq 0 \rightarrow PC = PC[63..18] + index \parallel 0^2$

dsll	Rd,Rt,imm	Desplazamiento lógico a Izda.	$Rd = Rt \ll imm$
dsrl	Rd,Rt,imm	Desplazamiento lógico a Dcha.	$Rd = Rt \gg imm$
dsra	Rd,Rt,imm	Desplazamiento aritm. a Dcha.	$Rd = Rt \gg imm$
dsllv	Rd,Rt,Rs	Desplazamiento lógico a izda.	$Rd = Rt \ll Rs_{(5..0)}$
dsrlv	Rd,Rt,Rs	Desplazamiento lógico a dcha.	$Rd = Rt \gg Rs_{(5..0)}$
dsrav	Rd,Rt,Rs	Desplazamiento aritmético a dcha.	$Rd = Rt \gg Rs_{(5..0)}$

movz	Rd,Rs,Rt	Movimiento condicional	Si $Rt=0 \rightarrow Rd = Rs$
movn	Rd,Rs,Rt	Movimiento condicional	Si $Rt \neq 0 \rightarrow Rd = Rs$
slt	Rd,Rs,Rt	Comparación entera con signo	Si $Rs < Rt \rightarrow Rd=1$ else $Rd = 0$
sltu	Rd,Rs,Rt	Comparación entera sin signo	Si $Rs < Rt \rightarrow Rd=1$ else $Rd = 0$
slti	Rd,Rs,imm	Comp. inmediata con signo	Si $Rs < \text{ext_sign}(imm) \rightarrow Rd=1$ else $Rd=0$
sltiu	Rd,Rs,imm	Comp. Inmediata sin signo	Si $Rs < \text{ext_sign}(imm) \rightarrow Rd=1$ else $Rd=0$
nop		Corresponde a sll r0,r0,0	No Operación
halt		¡No existe en MIPS64!	Vuelta al S.O.

add.d	Fd,Fs,Ft	Suma en coma flotante	$Fd = Fs + Ft$
sub.d	Fd,Fs,Ft	Resta en coma flotante	$Fd = Fs - Ft$
mul.d	Fd,Fs,Ft	Multiplicación en coma flotante	$Fd = Fs * Ft$
div.d	Fd,Fs,Ft	División en coma flotante	$Fd = Fs / Ft$

mov.d	Fd,Fs	Copia entre registros de c. f.	$Fd = Fs$
mtc1	Rt,Fs	Copia de registro entero a c.f.	$Fs[31..0] = Rt[31..0]$
mfc1	Rt,Fs	Copia de registro de c. f. a entero	$Rt = \text{ext_signo}(Fs[31..0])$
cvt.d.l	Fd,Fs	Convierte entero de 64 bits a c. f.	$Fd = \text{toDoubleFP}(Fs)$
cvt.l.d	Fd,Fs	Convierte de c. f. a entero de 64 bits	$Fd = \text{to64integer}(Fs)$

l.d	Ft,offset(Rs)	Carga número de 64 bits en c. f.	$Ft[0..63] = \text{MEM}[Rs + \text{offset}]$
s.d	Ft,offset(Rs)	Almacena número de 64 bits en c. f.	$\text{MEM}[Rs + \text{offset}] = Ft[0..63]$

c.lt.d	Fs,Ft	Establece código de condición	$CC = (Fs < Ft)$
c.le.d	Fs,Ft	Establece código de condición	$CC = (Fs \leq Ft)$
c.eq.d	Fs,Ft	Establece código de condición	$CC = (Fs = Ft)$
bc1f	offset	Salta si el c. c. es falso	Si $CC=0 \rightarrow \text{Saltar}$
bc1t	offset	Salta si el c. c. es cierto	Si $CC=1 \rightarrow \text{Saltar}$